

6강. 마이크로서비스 설계



이영곤



구분	이벤트 스토밍	도메인 주도 설계 (DDD)	업무기능 분해
접근 기준	이벤트 흐름	도메인 / 경계 (Bounded Context)	기능 / 프로세스
핵심 개념	이벤트 중심 협업 워크샵	도메인 모델링 기반 서비스 분리	기능 단위 분해
장점	- 빠른 이해 가능 - 이해관계자 참여 용이 - 숨은 요구사항 발견	- 경계가 명확함 - 비즈니스 정합성 높음 - 확장/유지보수 유리	- 적용이 쉬움 - 빠른 도출 가능 - 기존 시스템 활용 용이
단점	- 결과가 사람 의존적 - 구조 설계로 바로 연결 어려움	- 난이도 높음 - 시간/비용 큼 - 도메인 이해 필요	- 경계 모호 - 결합도 증가 위험 - 분산 모놀리스 가능
적합 상황	초기 분석 / 공통 이해 필요 시	복잡한 비즈니스 시스템	단순 시스템 / 빠른 도입
활용 전략	초기 탐색 도구 	최종 설계 기준 	보조적 분해 기준

■ 기존 소프트웨어 설계 방식의 문제점

- ❖ 최신 기술 중심적인 설계 반복
- ❖ 도메인에 대한 지식과 문제에 대한 이해 부족
- ❖ 개념의 분할 없이 계속해서 필요한 기능을 핵심 개념에 추가함
- ❖ 데이터에 종속적인 애플리케이션
- ❖ 모델링과 개발과의 불일치 발생(분석모델-구현모델, 구현모델-코드 간 연계 부족)

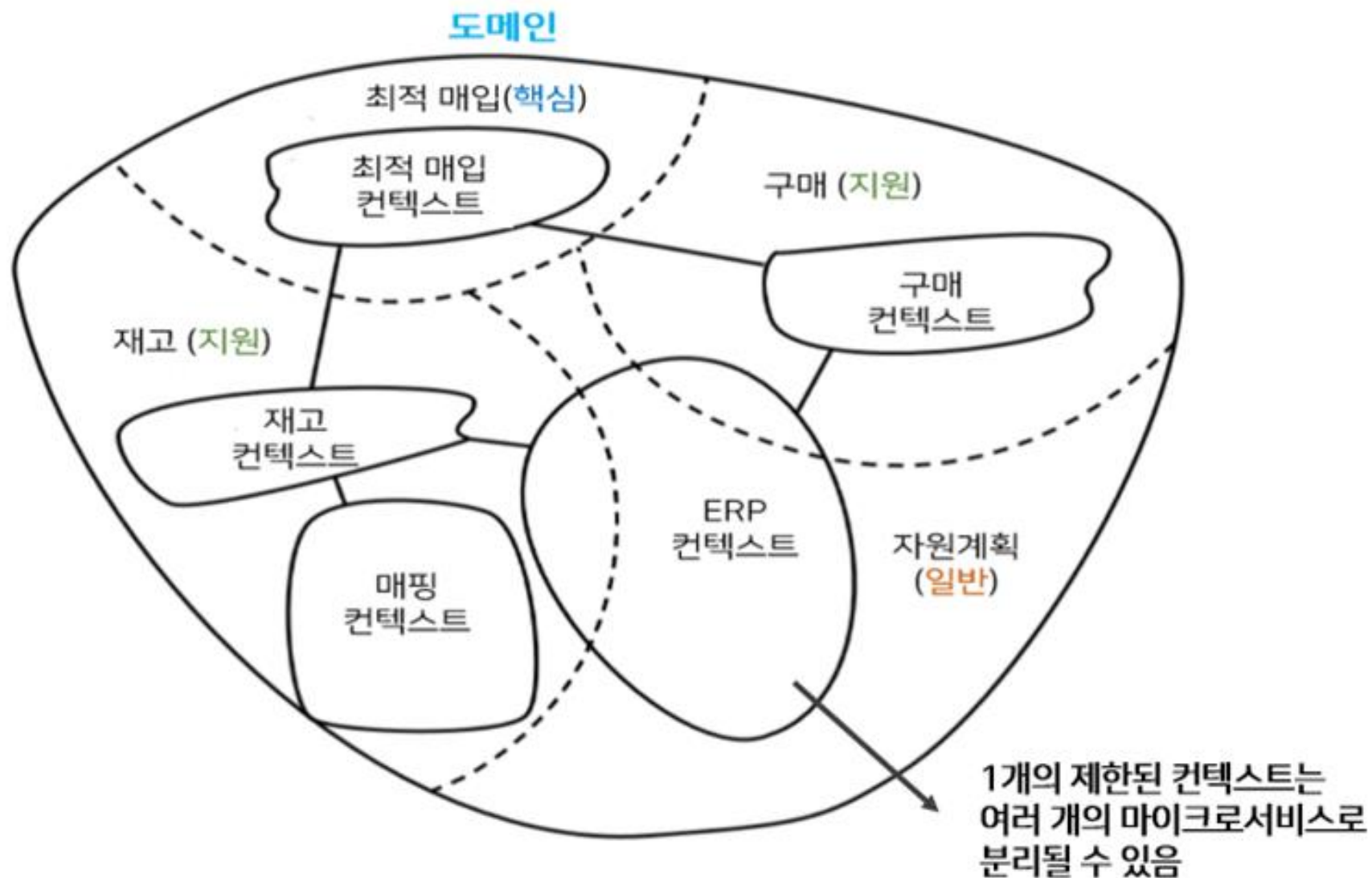
도메인 주도 설계

- 도메인 주도 설계는 모델링과 개발의 불일치 문제를 해결하는 데 필요
- 모든 참여자가 비즈니스나 현실의 문제를 개념적으로 표현한 도메인 모델을 작성
- 도메인 모델은 비즈니스나 현실의 문제, 즉 도메인을 개념적으로 표현
- 도메인 모델의 핵심 원칙은 도메인 모델링 수행 시 사용자, 도메인 전문가, 분석가, 개발자 등이 동일한 모습으로 도메인을 이해하고 도메인 지식을 공유하는 것



개발자들이
도메인 전문가, 분석가, 사용자
등과 함께 회의에 참여하고,
도메인과 모델을 정확하게 이해
해야 함

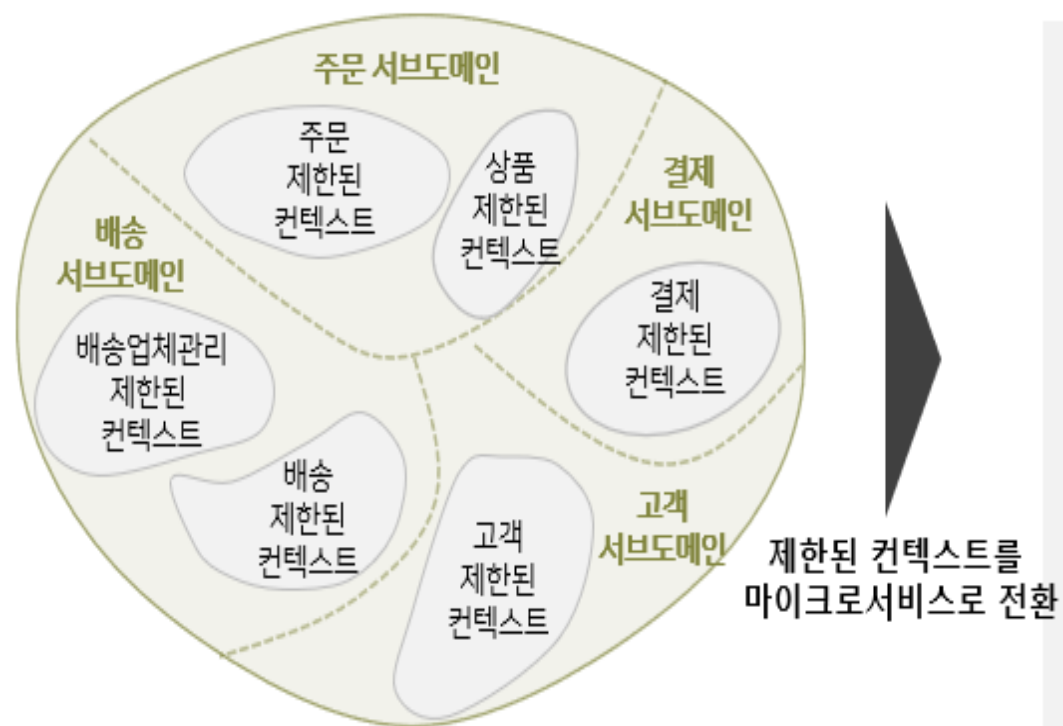
- 도메인이란 소프트웨어가 해결해야 할 문제 영역
 - 세부 업무 영역 또는 조직 단위의 서브도메인으로 구성
- 서브도메인은 도메인 내의 문제 해결을 위해 도메인을 여러 개로 나눈 것
- 서브도메인은 핵심, 지원, 일반 서비스 도메인으로 분류됨
 - 핵심 서브도메인 : 핵심적인 비즈니스 로직이 담긴 영역
 - 지원 서브도메인 : 핵심 서브 도메인을 기능적으로 보조할 수 있는 영역
 - 일반 서브도메인 : 범용적으로 사용될 수 있는 도메인



DDD 설계 원칙

- 바운디드 컨텍스트는 도메인의 문제를 해결하기 위한 솔루션 영역으로 도메인 모델이 존재하는 명시적인 경계
- 바운디드 컨텍스트는 전체 비즈니스 도메인을 여러 개의 서브 도메인으로 나눈 후, 서브 도메인 내 같은 컨텍스트를 경계로 구분하여 도출
- 서브 도메인과 바운디드 컨텍스트의 관계는 1:1이 이상적임. 1:N의 관계가 될 수도 있음
- 각각의 마이크로서비스는 가능한 독립적으로 도출되어야 함

DDD 개략적 사례



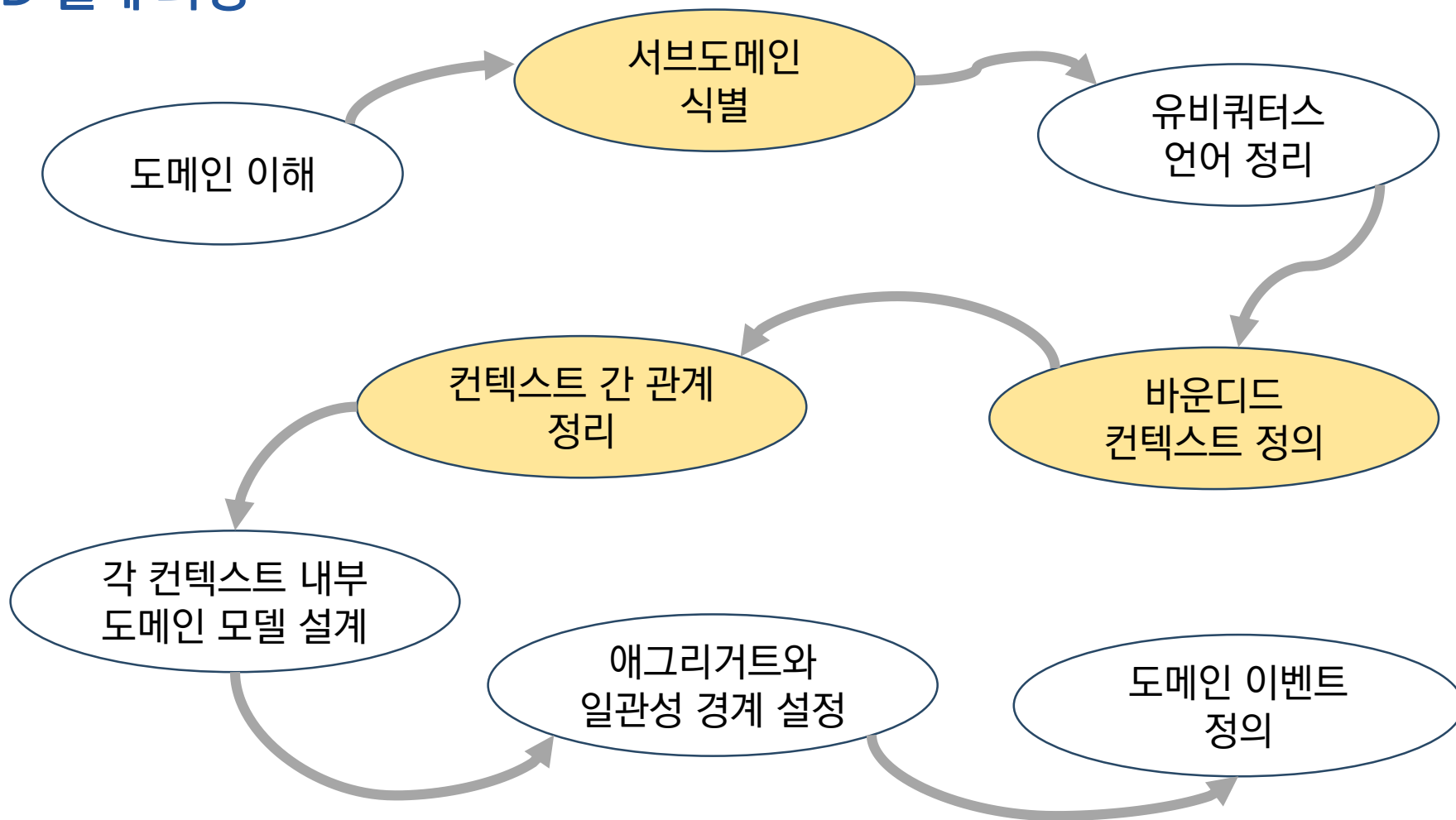
마이크로서비스 도출



도메인 주도 설계

구분	전략적 설계 (기본 설계)	전술적 설계 (상세 설계)
수행 단계	분석 단계	설계 단계
목적	마이크로서비스 도출	마이크로서비스 내부 구조 설계
주요 내용	- 제한된 컨텍스트 도출을 통한 마이크로서비스 정의 - 유비쿼터스 언어 사용 - 컨텍스트 맵 작성	- 애그리게이트 (Aggregate) - 엔티티 (Entity) - 값 객체 (Value Object) - 레포지토리 (Repository) - 서비스 (Service) - 도메인 이벤트 (Domain Event) - 모듈 (Module)

DDD 설계 과정



1. 문제 영역 이해

- ✓ DDD의 출발점은 기술이 아니라 도메인 이해
- ✓ 먼저 해야 할 일은 “무엇을 만들까?”가 아니라 “업무가 실제로 어떻게 돌아가나?”를 파악하는 것
- ✓ 이 단계에서 확인할 것들:
 - 핵심 업무가 무엇인가
 - 누가 어떤 일을 하는가
 - 어떤 규칙과 제약이 있는가
 - 어떤 예외 상황이 자주 발생하는가
 - 어디서 변경 요청이 자주 발생하는가

2. 유비쿼터스 언어(Ubiquitous Language) 정리

- ✓ 같은 도메인내에서는 같은 언어를 사용해야 함
- ✓ 같은 말을 서로 다르게 쓰거나, 다른 말을 같은 뜻으로 쓰면 설계가 계속 흔들림
 - “주문 접수”와 “주문 생성”은 같은가?
 - “출고”와 “배송 시작”은 같은가?
 - “취소”와 “환불”은 같은가?
 - “회원”과 “고객”은 같은가?

3. 서브도메인 식별

- ✓ 전체 비즈니스는 보통 하나의 도메인도 여러 하위 영역으로 나눌 수 있음
- ✓ 서브 도메인:
 - 핵심 도메인(Core Domain)
 - 지원 도메인(Supporting Domain)
 - 일반 도메인(Generic Domain)
- ✓ 예를 들어, 일반 B2B에서,
 - 핵심 도메인: 주문 정책, 가격 정책, 프로모션
 - 지원 도메인: 배송, 정산
 - 일반 도메인: 로그인, 알림, 파일 업로드

4. 바운디드 컨텍스트(Bounded Context) 정의

- ✓ 같은 용어와 규칙이 일관되게 유지되는 경계를 정의하는 과정
- ✓ 예를 들어, 상품, 재고, 가격이 모두 품목코드를 쓰더라도, 그 안에서 “품목”의 의미가 조금씩 다를 수 있음.
 - 상품관리에서 품목 = 판매/관리 대상 마스터
 - 재고에서 품목 = 재고 수불의 대상
 - 가격에서 품목 = 가격 적용 대상
 - 같은 품목이라도 컨텍스트마다 보는 관점과 규칙이 다름
- ✓ 따라서, 모든 것을 하나의 품목 모델로 합치면 안 되고, 어디까지 같은 모델을 공유할지 경계를 정해야 함



4. 바운디드 컨텍스트(Bounded Context) 정의 규칙

- ✓ 용어의 의미가 같아야 함
- ✓ 예를 들어, 거래처라는 말이
 - 거래처 관리에서는 고객/매입처 공통 마스터
 - 여신에서는 신용한도 평가 대상
 - 물류에서는 배송지와 출고처리 기준 대상
- ✓ 판단 질문
 - 같은 단어를 같은 의미로 말할 수 있는가?
 - 같은 속성과 같은 규칙으로 설명 가능한가?

4. 바운디드 컨텍스트(Bounded Context) 정의 규칙

- ✓ 핵심 비즈니스 규칙이 같아야 함.
 - 한 컨텍스트 안에서는 규칙의 일관성 보장

가격 컨텍스트

- ✓ 가격은 기간별로 바뀐다
- ✓ 고객별 예외가격이 가능하다
- ✓ 브랜드/채널 정책을 따른다

재고 컨텍스트

- ✓ 가용재고 계산이 중요하다
- ✓ 로케이션 이동과 수불 이력이 중요하다

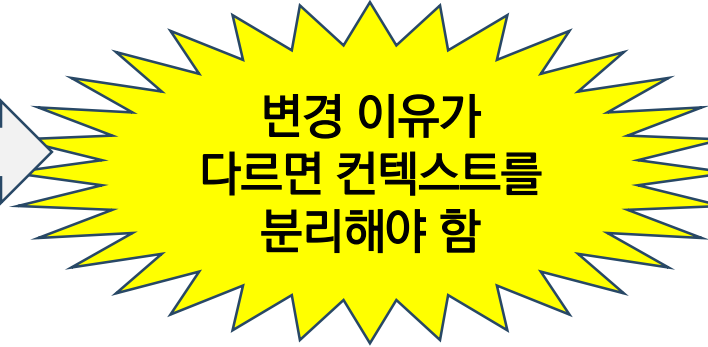
- ✓ 가격 컨텍스트와 재고 컨텍스트는 같은 품목을 참조하더라도 규칙이 완전히 다름
⇒ 하나의 컨텍스트로 묶으면 안됨

4. 바운디드 컨텍스트(Bounded Context) 정의 규칙

- ✓ 바운디드 컨텍스트내에서는 모델 변경 이유가 같아야 함
 - 한쪽은 영업정책이 바뀌어서 수정
 - 다른 한쪽은 창고운영 방식이 바뀌어서 수정이라면 사실 서로 다른 모델임

판단 기준

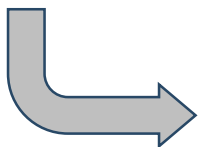
- ✓ “이 모델은 왜 바뀌는가?”
 - 가격은 할인정책/프로모션 때문에 바뀜
 - 재고는 물류 프로세스/창고 운영 때문에 바뀜
 - 거래처는 거래조건/조직/주소 때문에 바뀜



변경 이유가
다르면 컨텍스트를
분리해야 함

4. 바운디드 컨텍스트(Bounded Context) 정의 규칙

- ✓ 트랜잭션 일관성 경계가 같아야 한다.
- ✓ 하나의 컨텍스트 안에서는 가능하면 강한 일관성을 요구하는 작업이 모여야 함
 - 예: 품목 등록 시 바코드와 기본 분류를 함께 저장해야 한다
→ 같은 컨텍스트 가능
 - 예: 가격 변경과 재고 이동이 항상 하나의 트랜잭션이어야 하나?
→ 대개 아니다



- ✓ 가격과 재고는 다른 컨텍스트 가능성 높음

4. 바운디드 컨텍스트(Bounded Context) 정의 규칙

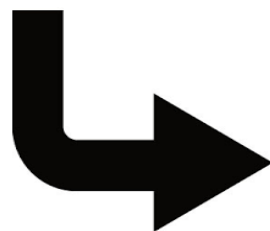
- ✓ 담당 조직/의사결정권자가 같아야 함
- ✓ 조직도만 기준으로 삼으면 안 되지만, 실제 책임 주체는 매우 중요한 힌트
 - 분류/브랜드/품목 기준정보는 상품MD 조직
 - 가격은 영업기획/정책 조직
 - 재고/로케이션은 물류 운영 조직



- ✓ 책임자와 의사결정 구조가 다르면 분리 가능성이 높음
- ✓ 단, 조직은 보조 신호이고, 핵심은 용어/규칙/변경 이유

5. 컨텍스트 맵(Context Map) 작성

- ✓ 정의: 각 컨텍스트가 서로 어떻게 연결되고, 누가 누구에게 의존하며, 어떤 방식으로 협력하는지를 보여주는 지도
- ✓ 구성요소
 - 바운디드 컨텍스트: 하나의 모델, 책임, 데이터 소유권
 - 관계: 컨텍스트간 연결방식
 - 관계 유형: Customer-Supplier, Conformist, Anti-Corruption Layer, Shared Kernel



- ✓ 경계 명확화
- ✓ 데이터 소유권 정의
- ✓ 의존성 관리
- ✓ 통합 방식 결정

5. 컨텍스트 맵(Context Map) 관계 유형

✓ Customer-Supplier 관계

- 상류가 모델/데이터를 제공하고, 하류가 이를 사용
- 사례: Catalog → Pricing, Catalog → Inventory

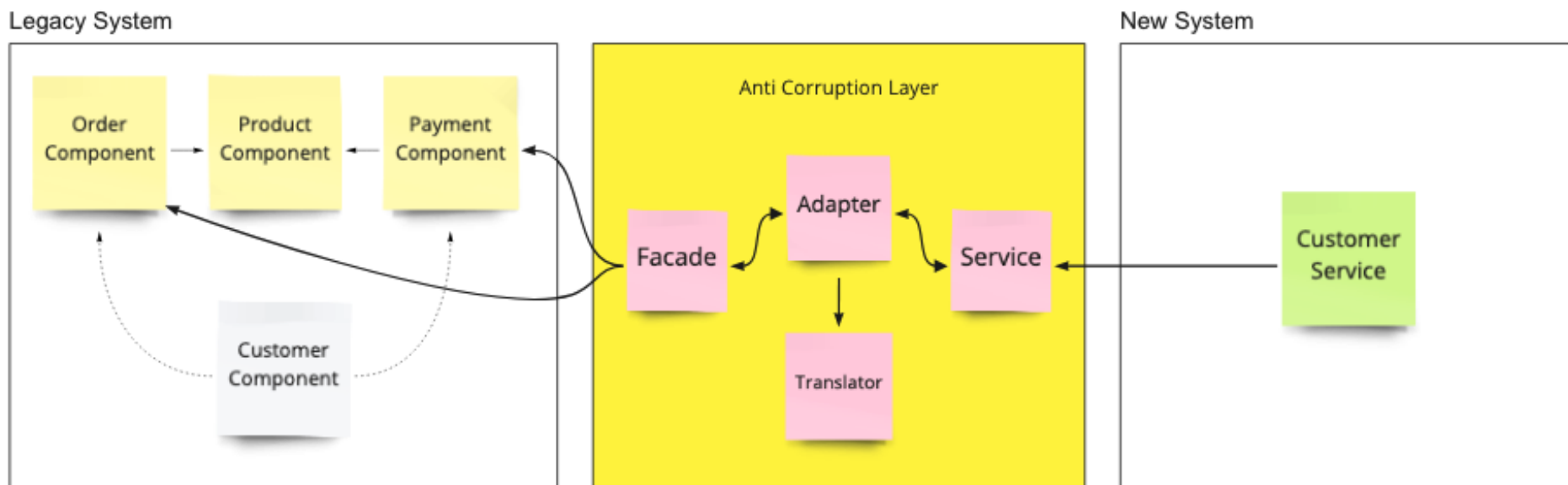
✓ Conformist 관계

- 하류가 상류의 모델을 거의 그대로 따름
- 예: Inventory는 Catalog의 상품 식별체계를 따를 가능성이 큼, Credit은 Partner의 거래처 식별체계를 따를 가능성이 큼

5. 컨텍스트 맵(Context Map) 관계 유형

✓ Anti-Corruption Layer

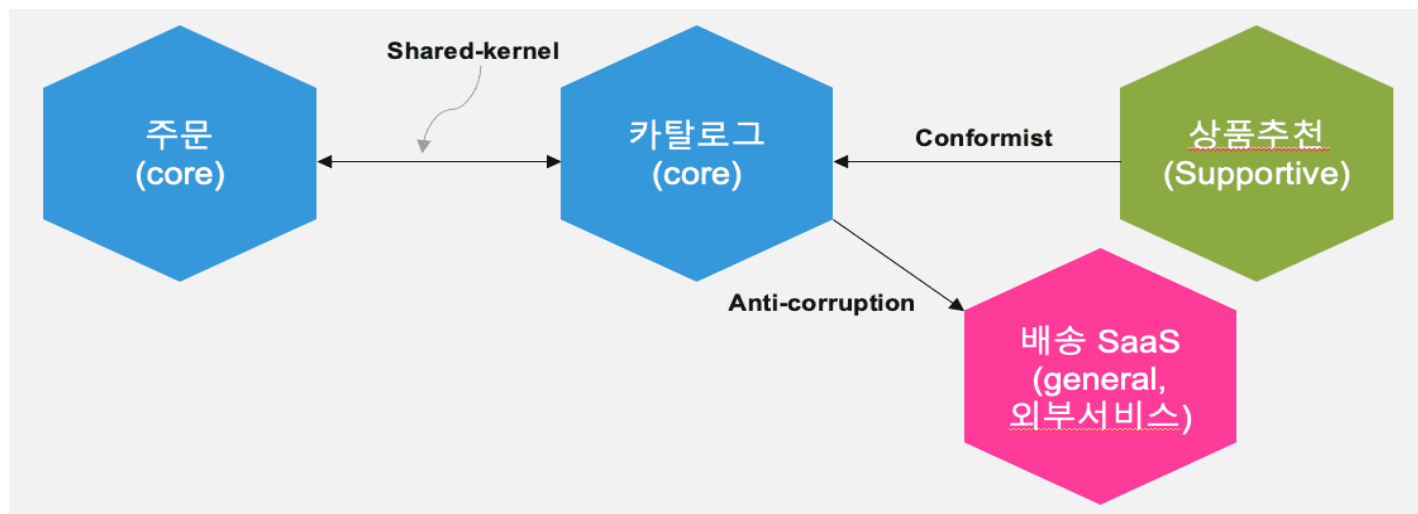
- 레거시 ERP 테이블 구조를 바로 받아들이면 모델이 오염될 때 사용
- 예: 현재 레거시 ERD → 새 Catalog, 현재 레거시 ERD → 새 Pricing, 현재 레거시 ERD → 새 Partner
- 신규 컨텍스트는 레거시의 코드/컬럼 구조를 그대로 노출하지 말고 ACL을 통해 받아야 함



5. 컨텍스트 맵(Context Map) 관계 유형

✓ Shared Kernel

- 여러 바운디드 컨텍스트가 공통으로 사용하는 “모델의 일부”를 함께 관리
- 예: 공통 Value Object, 공통 Entity의 일부 속성, 도메인 규칙 (validation logic), 공통 Enum / 코드값, 유비쿼터스 언어 정의
- 단, 공유 범위는 아주 작아야 함. 공유가 커지면 다시 하나의 거대한 레거시 모델이 됨



5. 컨텍스트 맵(Context Map) 관계 유형: 기타

관계 유형	정의	역할 구조	방향성	상태 변경 여부	통신 방식	언제 사용하는가
Partnership	두 컨텍스트가 공동으로 모델과 규칙을 협의하며 함께 진화	$A \leftrightarrow B$ (동등)	양방향	둘 다 가능	API / 이벤트 / 직접 협업	경계가 아직 명확하지 않거나 규칙이 강하게 얽혀 있을 때
Published Language	공통 메시지/이벤트 스키마를 정의하여 여러 컨텍스트가 공유	Publisher → Subscribers	단방향 (다수)	Publisher에서만	이벤트 (Kafka 등)	느슨한 결합, 확장성 필요, 다수 소비자 존재
Policy (Policy Provider)	한 컨텍스트가 업무 규칙/판단 기준을 제공	Policy → Executor	단방향	없음 (판단만)	API / 동기 호출	승인, 검증, 제한 판단이 필요할 때
Execution (Executor)	다른 컨텍스트의 정책/요청에 따라 실제 작업 수행	Policy ← Execution	단방향 (의존)	있음 (핵심)	내부 처리 / API / 이벤트	실제 데이터 변경, 물리적 처리

6. 핵심 모델 설계

- ✓ 각 바운디드 컨텍스트 안에서 도메인 모델을 설계
- ✓ 주요 구성 요소:
 - Entity
 - Value Object
 - Aggregate
 - Domain Service
 - Repository
 - Domain Event

7. 애그리거트(Aggregate) 경계 설정

- ✓ 애그리거트는 데이터 묶음이 아니라, 일관성을 보장하는 변경 단위
- ✓ 한 트랜잭션 안에서 강하게 일관성을 유지해야 하는 범위를 정하는 것
- ✓ 애그리게이트 경계 = 일관성 + 트랜잭션 + 동시성 제어 단위
 - 예: 주문금액 = 주문항목 합계 ➡ 동시에 맞아야 함



상황	선택
즉시 일관성 필요	애그리게이트 내부 처리
서비스 간 협력	SAGA
실패 허용 + 보상 가능	SAGA
절대 깨지면 안됨	애그리게이트

8. 도메인 이벤트 정의

- ✓ 컨텍스트 내부 또는 컨텍스트 간 협력을 위해 도메인 이벤트를 정의
 - OrderCreated
 - PaymentCompleted
 - InventoryReserved
 - ShipmentStarted
- ✓ 도메인 이벤트는 “무엇을 할지 명령”이 아니라, 이미 발생한 사실을 표현하는 것

9. 애플리케이션 계층과 인프라 계층 설계

- ✓ 도메인 모델이 정리되면 그것을 실제 애플리케이션 구조로 연결
- ✓ 보통 계층은 다음과 같음:
 - Presentation/API 계층
 - Application 계층
 - Domain 계층
 - Infrastructure 계층

10. 구현과 검증

- ✓ DDD는 문서 설계에서 끝나지 않고, 실제로 구현해 보면서 모델이 맞는지 계속 검증해야 함
- ✓ 검증 포인트:
 - 업무 규칙이 모델 안에 자연스럽게 표현되는가
 - 서비스 책임이 명확한가
 - 컨텍스트 경계가 과도하게 엮이지 않는가
 - 조회/수정 흐름이 현실적으로 동작하는가
 - 도메인 이벤트 흐름이 비즈니스와 맞는가
 - 이 단계에서 자주 일어나는 일이 경계 재조정

11. 리팩토링과 지속적 개선

- ✓ DDD는 한 번 설계하고 끝나는 방식이 아니고, 도메인 이해가 깊어질수록 모델도 진화해야 함
 - 예를 들어, 처음엔 주문 컨텍스트 안에 넣었던 할인 정책이,
 - 나중에 보니 프로모션 컨텍스트로 분리되어야 할 수도 있음
 - 또는 배송과 출고를 하나로 봤다가,
 - 실제 운영상 별도 팀과 별도 규칙을 가지면 분리해야 할 수도 있음
- ✓ 즉, DDD 프로세스는 선형 완료형이 아니라 반복적 개선형

비즈니스 중심으로 경계를 나눔

- ✓ DDD의 가장 큰 장점은 서비스 분리가 기술 구조가 아니라 업무 의미를 반영
 - 예를 들어 주문, 재고, 결제는 화면상 한 번에 보일 수 있어도 실제 업무 규칙은 다름
- ✓ DDD는 이런 차이를 반영해 각 서비스의 책임을 명확히 나눔
 - 서비스 책임이 명확해지고
 - 변경 영향 범위가 줄고
 - 이 기능은 어느 서비스 책임인가가 분명해짐

높은 응집도와 낮은 결합도를 만듦

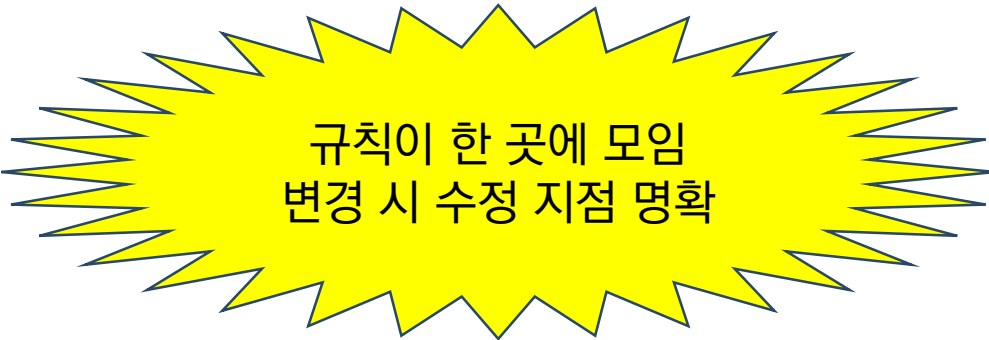
- ✓ DDD는 관련 있는 규칙과 데이터를 한 컨텍스트 안에 모으도록 유도
- ✓ 예를 들면:
 - 주문 할인 규칙, 주문 상태 전이, 주문 취소 조건
⇒ 주문 컨텍스트
 - 결제 승인 규칙은
⇒ 결제 컨텍스트

데이터 소유권을 명확히 함

- ✓ DDD 기반 MSA에서는 보통 서비스별 DB 소유권을 강조
 - Order Service → 주문 데이터 소유
 - Payment Service → 결제 데이터 소유
 - Inventory Service → 재고 데이터 소유

복잡한 업무 규칙을 쉽게 모델링

- ✓ 단순 CRUD 시스템보다, 규칙이 많고 상태 변화가 복잡한 업무에서 유리
 - 주문 상태 변경 규칙
 - 환불 가능 조건
 - 회원 등급별 혜택
 - 출고 가능 여부 판단
- ✓ 규칙이 많은 시스템은 테이블 중심 설계보다 도메인 모델 중심 설계가 더 유지보수에 유리



규칙이 한 곳에 모임
변경 시 수정 지점 명확

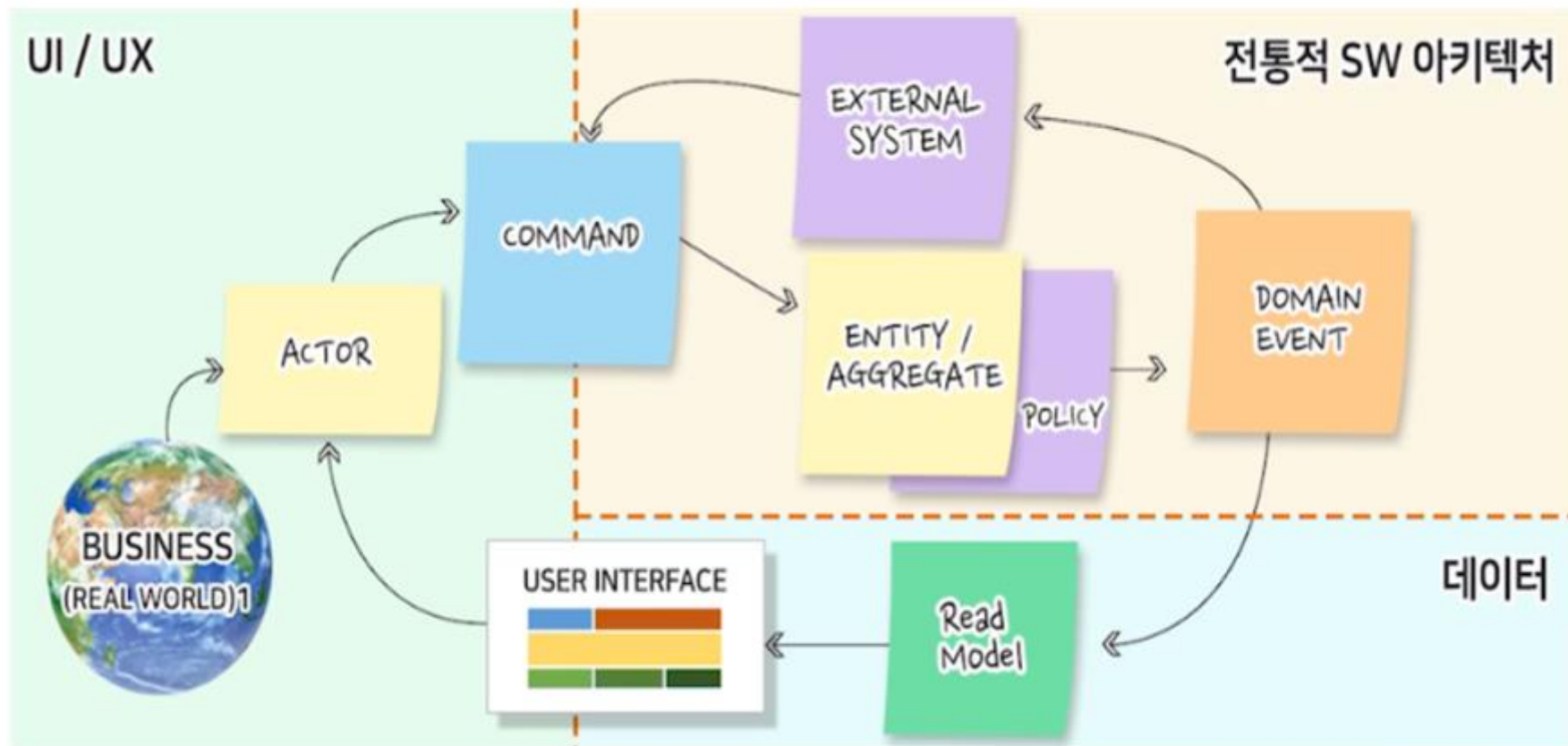
설계 난이도 높음

- ✓ DDD는 단순히 클래스 잘 나누는 방법이 아니라 도메인을 깊게 이해하고 경계를 설계하는 작업
- ✓ 도메인에 대한 깊은 이해 필요
 - 예를 들어, 어디까지를 주문 도메인으로 볼 것인가?
 - 할인은 주문인가 프로모션인가?
 - 포인트는 결제인가 회원인가?
 - 배송비 계산은 주문인가 배송인가?

과도한 모델링 위험

- ✓ DDD를 적용한다고 하면서 실제로는 작은 기능까지 모두 복잡한 객체 모델로 만들고, 패턴을 남용하는 경우 발생
 - 예:단순 CRUD인데 Aggregate, Domain Service, Factory, Repository를 과하게 도입
- ✓ 실익 없는 이벤트 남발
- ✓ 너무 잘게 분리된 마이크로서비스

- 이벤트 스토밍은 이벤트(Event)와 브레인스토밍(Brain Storming)의 합성어로 도메인의 이벤트를 통해 마이크로서비스를 도출하기 위한 브레인스토밍 활동으로 다음과 같은 특징이 있다.
 - ❖ 사전에 훈련된 지식과 도구 없이 수행 가능
 - ❖ 단시간 내 업무 흐름을 표현하는 모델을 만들 수 있음
 - ❖ 복잡한 시스템의 동작과 결과를 시각적으로 표현할 수 있음
 - ❖ 협력을 통해 업무 이해도 향상, 오류 및 누락된 개념 파악
 - ❖ 제한된 컨텍스트와 마이크로서비스를 빠르게 도출
 - ❖ 디자인과 코딩으로의 원활한 전환 가능

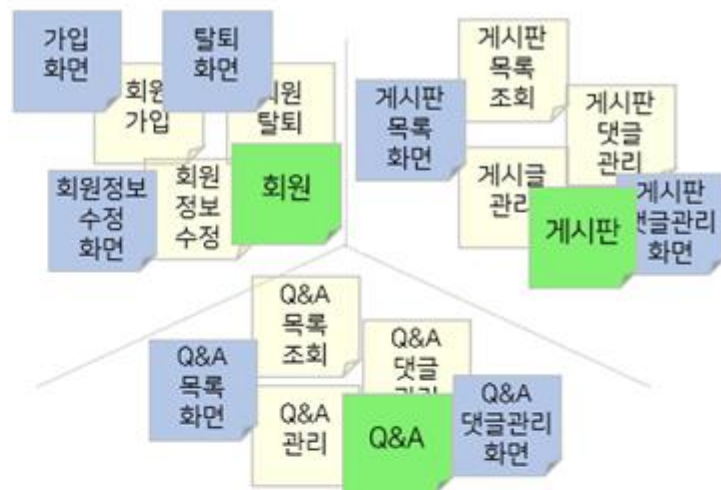




이벤트스토밍을 통한 마이크로서비스 도출



애그리게이트(엔티티) 도출 예시



엔티티에서 제한된 경계 도출 예시



※ 1개 팀에 의해 원활하게 운영하기 위해 마이크로서비스를 합치거나 분리할 수 있음

마이크로서비스 도출



이벤트 스토밍 결과

업무기능 분해를 통한 마이크로서비스 도출

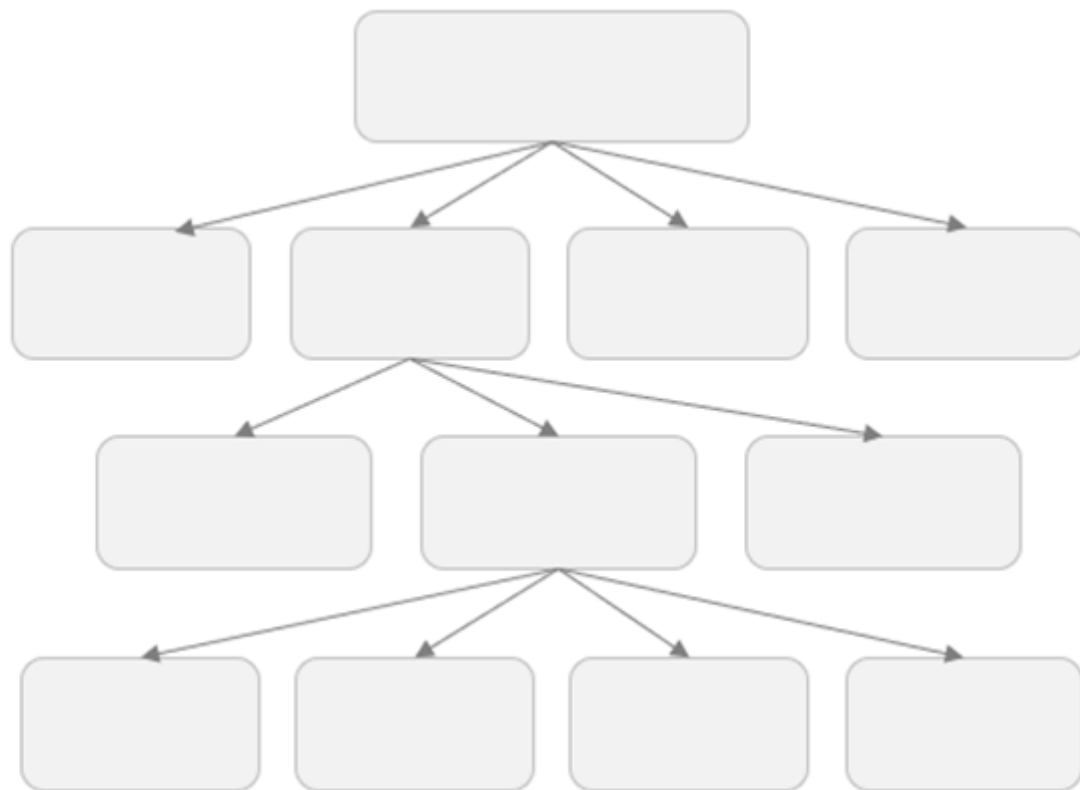
- 업무 프로세스가 잘 정리되어 있거나 기존의 정보시스템이 존재하는 경우, 업무기능을 분해함으로써 마이크로서비스를 도출할 수 있음
 - ❖ 비즈니스 업무를 분석하여 더 작은 업무로 나누고 그들 간의 계층구조 및 업무 간의 순서와 의존성을 분석하는 작업
 - ❖ 정보시스템을 구축하는 가장 최하위 단위의 프로세스를 도출하기에 유용하며 기관이 보유하고 있는 업무 프로세스 관련 자료 또는 애플리케이션의 맵 및 체계도를 활용
 - ❖ 마이크로서비스는 애플리케이션의 기능체계를 토대로 기능 간 응집도와 결합도, 독립적 개발 및 배포, 데이터 분리 가능성 등을 고려하여 도출

레벨 1.
기능영역

레벨 2.
업무기능

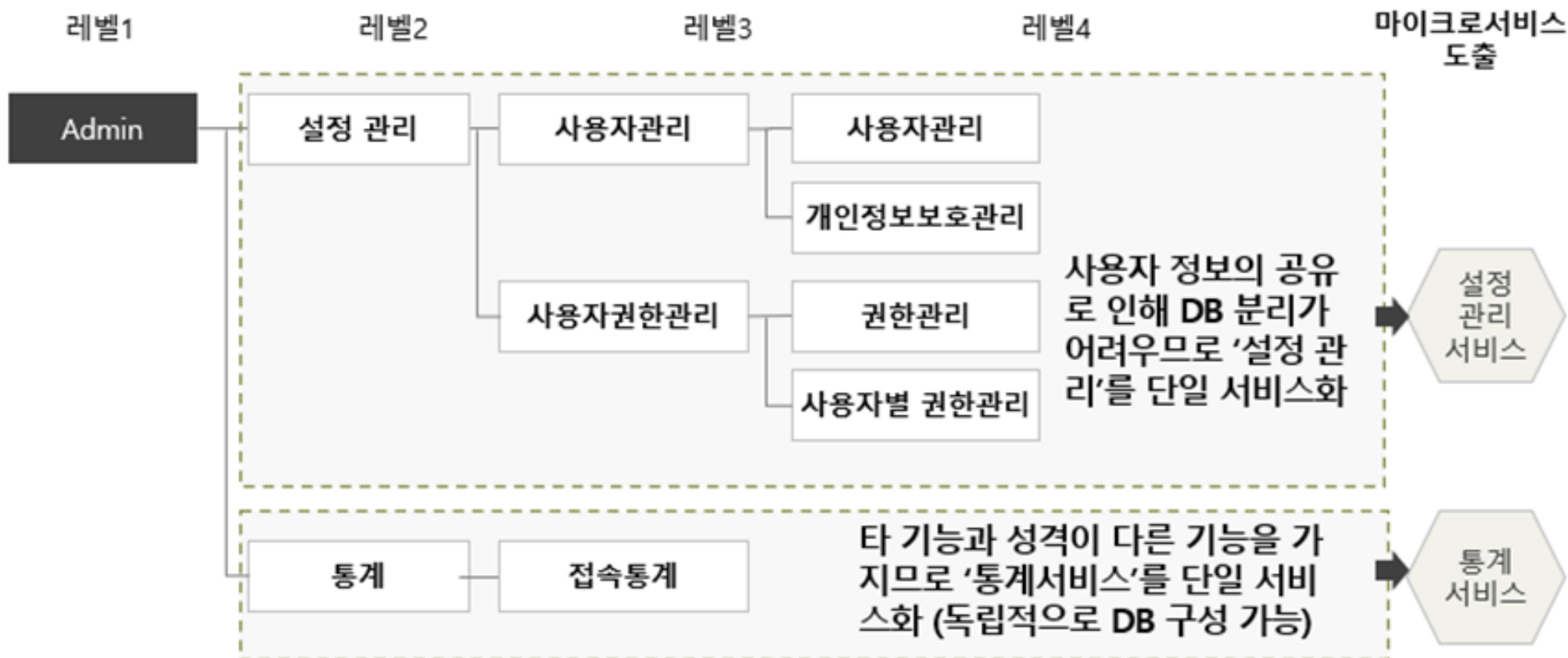
레벨 3.
업무프로세스

레벨 4.
단위프로세스

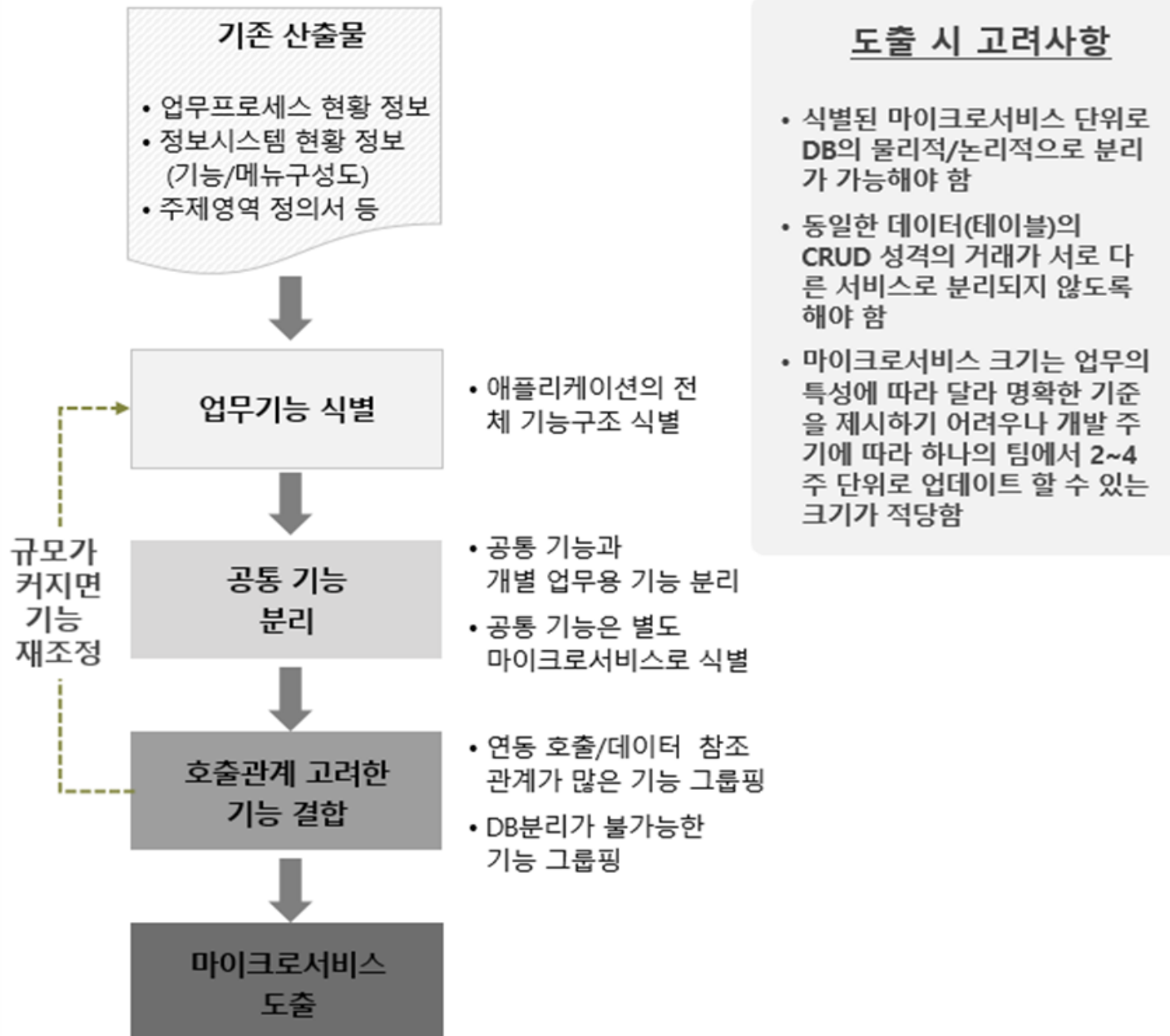


- 업무기능 분해를 통한 마이크로서비스 도출할 때에는 다음의 식별 원칙을 준수한다.
 - ❖ 애플리케이션의 하위 응용기능 단위를 마이크로서비스 단위로 식별. 이는 결합도에 따라 1개 혹은 복수의 기능으로 마이크로서비스를 구성할 수 있음.
 - ❖ 마이크로서비스는 독립적인 하나의 기능을 가지며, 높은 응집도와 타 서비스와는 낮은 결합도를 갖도록 구성해야 함. 구현 가능한 공유 메소드는 마이크로서비스 내에 직접 구현하고, 공통 컴포넌트는 별도의 서비스로 구현
 - ❖ 독립적으로 개발, 배포 및 데이터(DB) 분리가 가능한 단위

업무기능 분해를 통한 마이크로서비스 도출

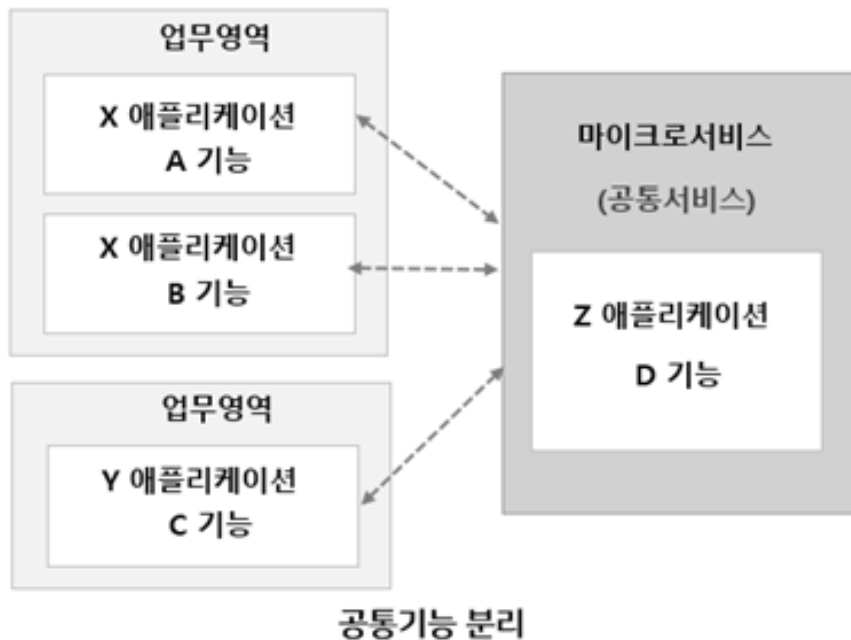


업무기능 분해를 통한 마이크로서비스 도출



- 식별된 업무기능은 공통서비스로 분리하여 코드의 중복성을 낮추어야 함
 - ❖ 공통 마이크로서비스의 크기가 너무 커지지 않도록 해야 함
- 하나의 마이크로서비스 내부에서 업무처리가 완료될 수 있도록 독립성 보장
- 내부 기능 간 상호 참조 및 공유가 빈번한 경우는 하나의 마이크로서비스로 결합

업무기능 분해를 통한 마이크로서비스 도출

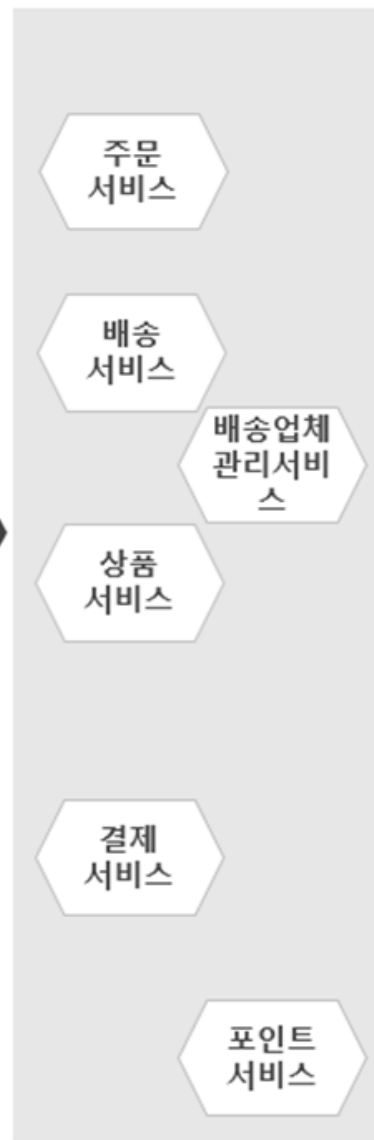


업무기능 분해를 통한 마이크로서비스 도출

재조정된 업무기능 분해도



마이크로서비스 도출



업무기능 분해를 통한 마이크로서비스 도출

홈페이지 기능 분해도



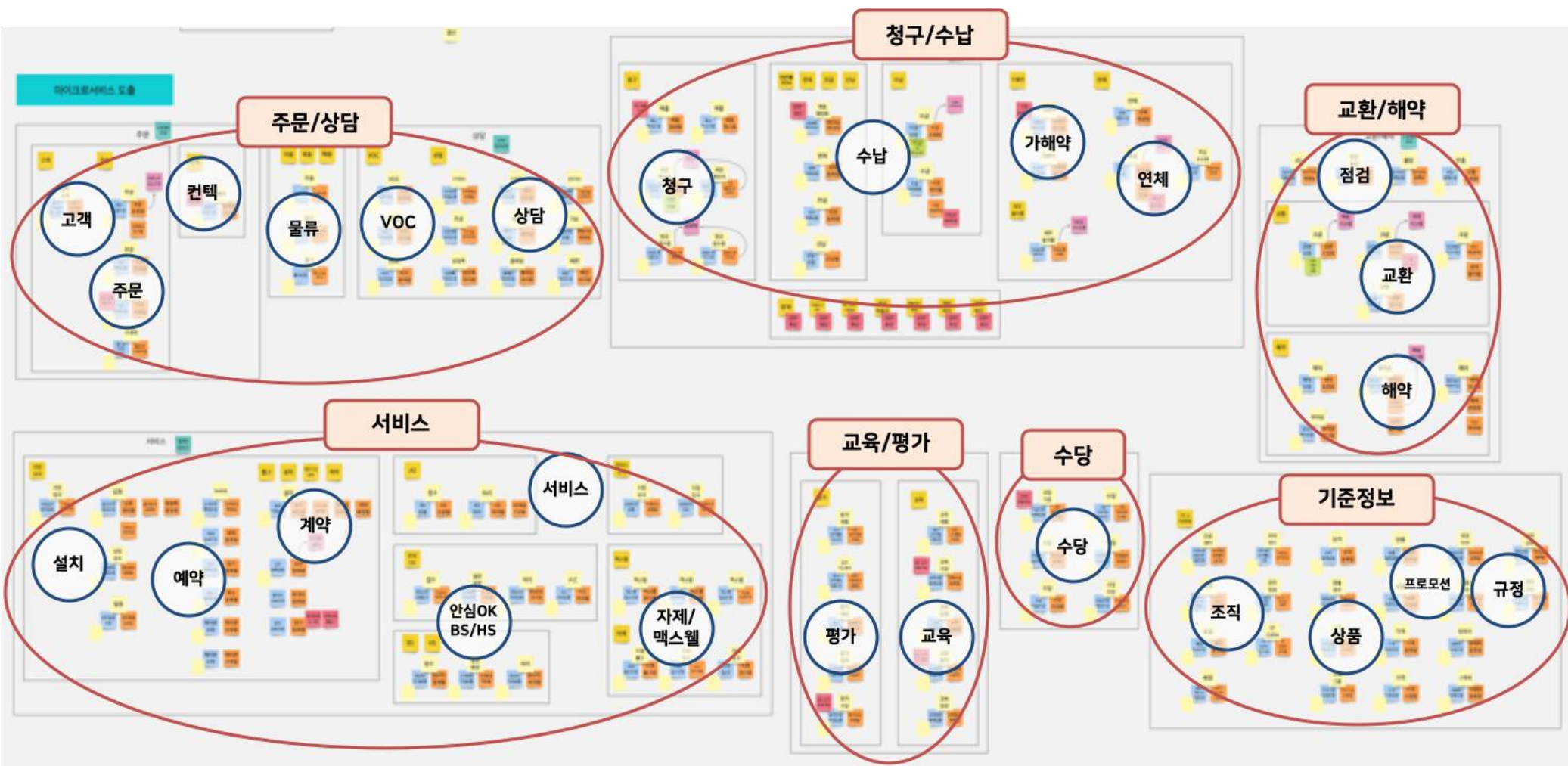
마이크로서비스 도출



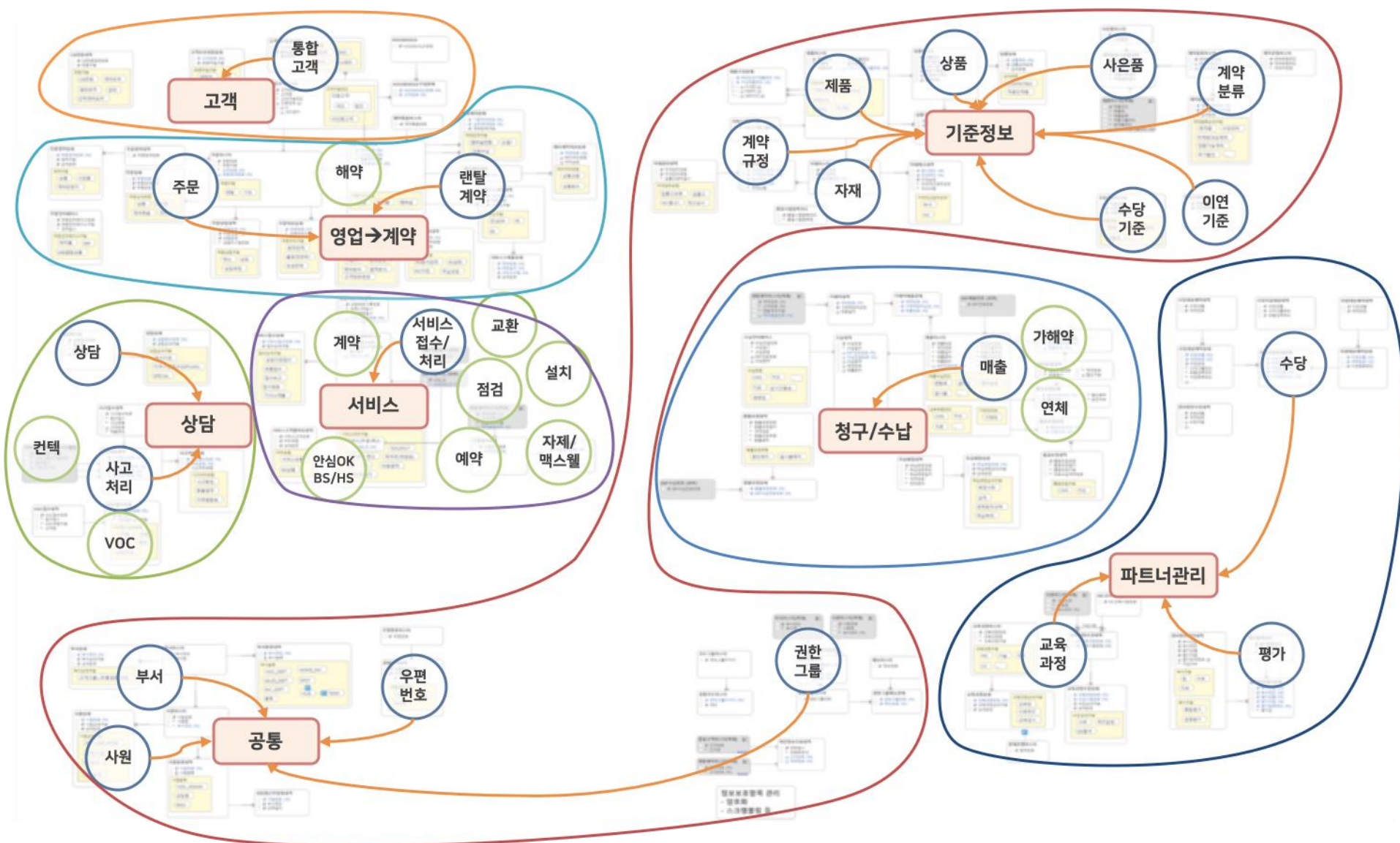
마이크로서비스 설계 사례(bounded context)



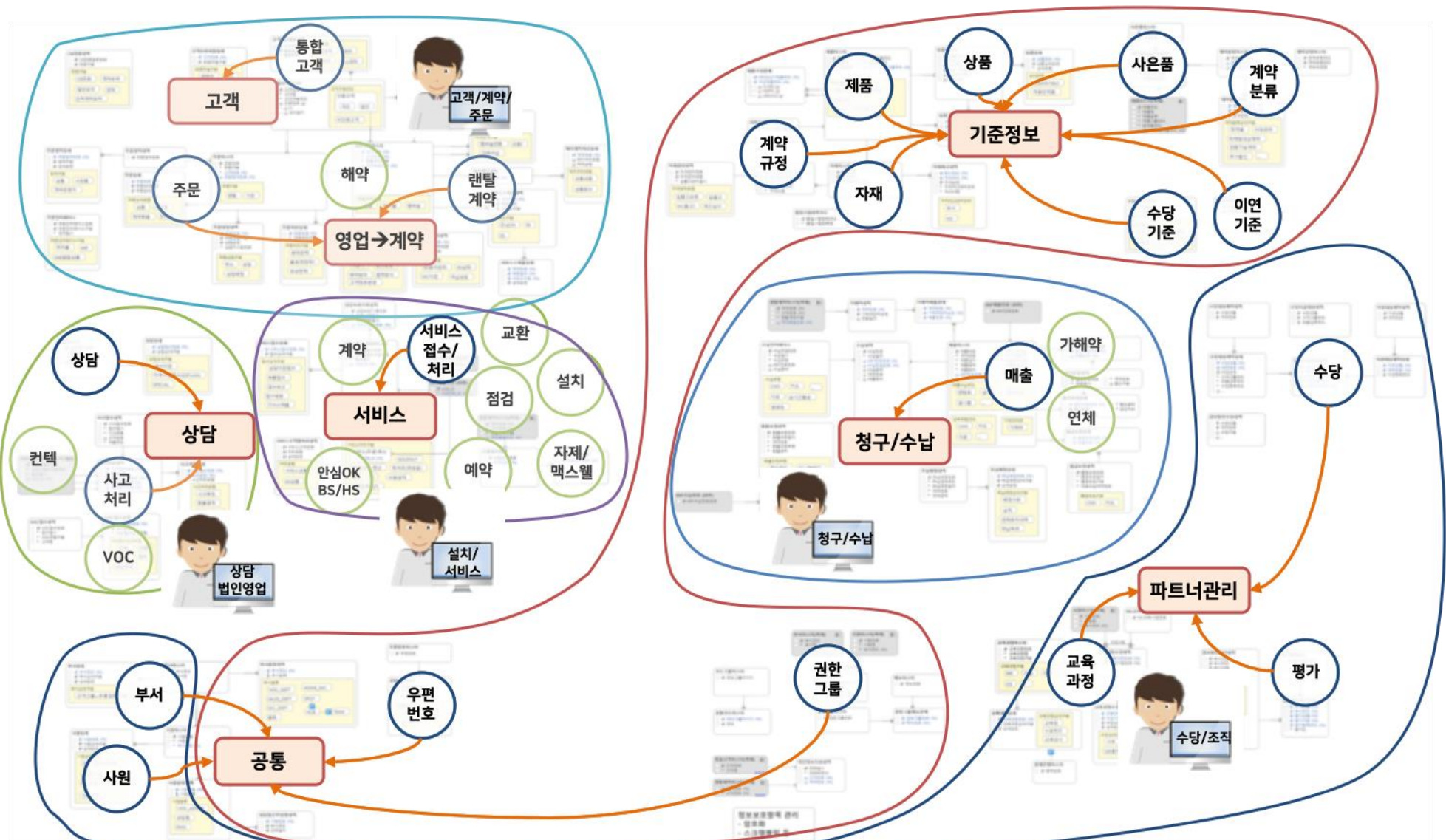
마이크로서비스 설계 사례(MS 정의)



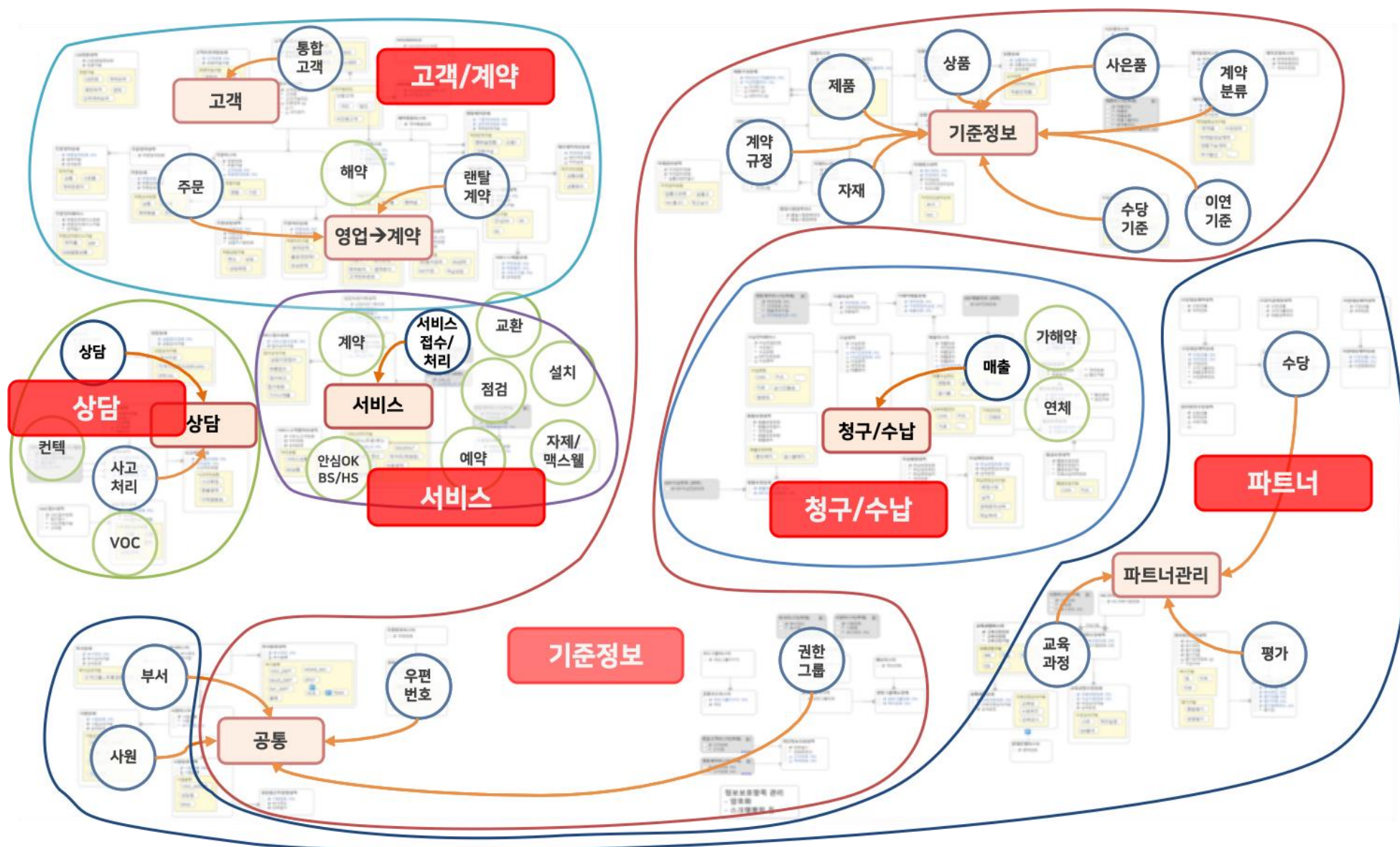
마이크로서비스 설계 사례(논리 ERD 분석)



마이크로서비스 설계 사례(MS 후보 검토)



마이크로서비스 설계 사례(최종 MS 도출)



Q & A